第4章 串

【考纲内容】

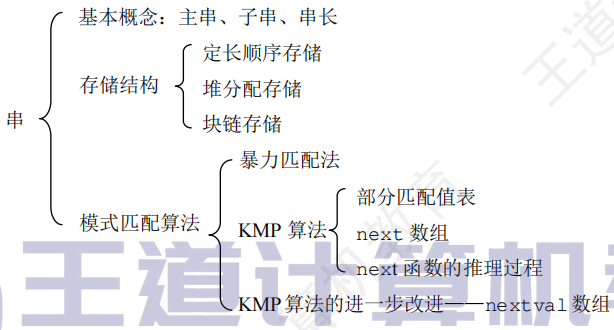
字符串模式匹配

扫一扫



视频讲解

【知识框架】



【复习提示】

本章是统考大纲第 6 章内容，采纳读者建议单独作为一章，大纲只要求掌握字符串模式匹配，重点掌握 KMP 匹配算法的原理及 next 数组的推理过程，手工求 next 数组可以先计算出部分匹配值表然后变形，或根据公式来求解。了解 nextval 数组的求解方法。

*4.1 串的定义和实现[①]

字符串简称串，计算机上非数值处理的对象基本都是字符串数据。我们常见的信息检索系统（如搜索引擎）、文本编辑程序（如 Word）、问答系统、自然语言翻译系统等，都是以字符串数据作为处理对象的。本章详细介绍字符串的存储结构及相应的操作。

4.1.1 串的定义

串（string）是由零个或多个字符组成的有限序列。一般记为

$$S='a_1a_2\cdots a_n' \quad (n\geqslant 0)$$

其中，S 是串名，单引号括起来的字符序列是串的值；a_i 可以是字母、数字或其他字符；串中字符的个数 n 称为串的长度。$n=0$ 时的串称为空串（用 $\varnothing$ 表示）。

串中任意多个连续的字符组成的子序列称为该串的子串，包含子串的串称为主串。某个字

① 本节不在统考大纲范围，仅供学习参考。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

符在串中的序号称为该字符在串中的位置。子串在主串中的位置以子串的第 1 个字符在主串中的位置来表示。当两个串的长度相等且每个对应位置的字符都相等时，称这两个串是相等的。

例如，有串 A='China Beijing'，B='Beijing'，C='China'，则它们的长度分别为 13、7 和 5。B 和 C 是 A 的子串，B 在 A 中的位置是 7，C 在 A 中的位置是 1。

注意，由一个或多个空格（空格是特殊字符）组成的串称为空格串（空格串不是空串），其长度为串中空格字符的个数。

串的逻辑结构和线性表极为相似，区别仅在于串的数据对象限定为字符集。在基本操作上，串和线性表有很大差别。线性表的基本操作主要以单个元素作为操作对象，如查找、插入或删除某个元素等；而串的基本操作通常以子串作为操作对象，如查找、插入或删除一个子串等。

4.1.2 串的基本操作

- StrAssign(&T,chars)：赋值操作。把串 T 赋值为 chars。
- StrCopy(&T,S)：复制操作。由串 S 复制得到串 T。
- StrEmpty(S)：判空操作。若 S 为空串，则返回 TRUE，否则返回 FALSE。
- StrCompare(S,T)：比较操作。若 S>T，则返回值>0；若 S=T，则返回值=0；若 S<T，则返回值<0。
- StrLength(S)：求串长。返回串 S 的元素个数。
- SubString(&Sub,S,pos,len)：求子串。用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。
- Concat(&T,S1,S2)：串联接。用 T 返回由 S1 和 S2 联接而成的新串。
- Index(S,T)：定位操作。若主串 S 中存在与串 T 值相同的子串，则返回它在主串 S 中第一次出现的位置；否则函数值为 0。
- ClearString(&S)：清空操作。将 S 清为空串。
- DestroyString(&S)：销毁串。将串 S 销毁。

不同的高级语言对串的基本操作集可以有不同的定义方法。在上述定义的操作中，串赋值 StrAssign、串比较 StrCompare、求串长 StrLength、串联接 Concat 及求子串 SubString 五种操作构成串类型的最小操作子集，即这些操作不可能利用其他串操作来实现；反之，其他串操作（除串清除 ClearString 和串销毁 DestroyString 外）均可在该最小操作子集上实现。

4.1.3 串的存储结构

1. 定长顺序存储表示

类似于线性表的顺序存储结构，用一组地址连续的存储单元来存储串值的字符序列。在串的定长顺序存储结构中，为每个串变量分配一个固定长度的存储区，即定长数组。

```
#define MAXLEN 255      //预定义最大串长为 255
typedef struct{
    char ch[MAXLEN];    //每个分量存储一个字符
    int length;         //串的实际长度
}SString;
```

串的实际长度只能小于或等于 MAXLEN，超过预定义长度的串值会被舍去，称为截断。串长有两种表示方法：一是如上述定义描述的那样，用一个额外的变量 len 来存放串的长度；二是在串值后面加一个不计入串长的结束标记字符“\0”，此时的串长为隐含值。

在一些串的操作（如插入、联接等）中，若串值序列的长度超过上界 MAXLEN，约定用“截

配套课程公众号[考机研]
永久联系微信:4550060

断”法处理，要克服这种弊端，只能不限定串长的最大长度，即采用动态分配的方式。

2．堆分配存储表示

堆分配存储表示仍然以一组地址连续的存储单元存放串值的字符序列，但它们的存储空间是在程序执行过程中动态分配得到的。

```
typedef struct{
    char *ch;              //按串长分配存储区，ch 指向串的基地址
    int length;            //串的长度
}HString;
```

在 C 语言中，存在一个称为堆的自由存储区，并用 malloc()和 free()函数来完成动态存储管理。利用 malloc()为每个新产生的串分配一块实际串长所需的存储空间，若分配成功，则返回一个指向起始地址的指针，作为串的基地址，这个串由 ch 指针来指示；若分配失败，则返回 NULL。已分配的空间可用 free()释放掉。

上述两种存储表示通常为高级程序设计语言所采用。块链存储表示仅做简单介绍。

3．块链存储表示

类似于线性表的链式存储结构，也可采用链表方式存储串值。由于串的特殊性（每个元素只有一个字符），在具体实现时，每个结点既可以存放一个字符，又可以存放多个字符。每个结点称为块，整个链表称为块链结构。图 4.1(a)是结点大小为 4（每个结点存放 4 个字符）的链表，最后一个结点占不满时通常用“#”补上；图 4.1(b)是结点大小为 1 的链表。

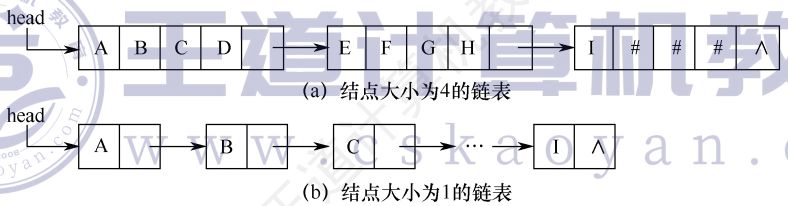


图 4.1　串值的链式存储方式

4.2　串的模式匹配

4.2.1　简单的模式匹配算法

模式匹配是指在主串中找到与模式串（想要搜索的某个字符串）相同的子串，并返回其所在的位置。这里采用定长顺序存储结构，给出一种不依赖于其他串操作的暴力匹配算法。

```
int Index(SString S,SString T){
    int i=1,j=1;
    while(i<=S.length&&j<=T.length){
        if(S.ch[i]==T.ch[j]){
            ++i;++j;           //继续比较后继字符
        }
        else{
            i=i-j+2;j=1;       //指针后退重新开始匹配
        }
    }
    if(j>T.length) return i-T.length;
    else return 0;
}
```

在上述算法中，分别用计数指针 i 和 j 指示主串 S 和模式串 T 中当前待比较的字符位置。

配套课程公众号[考机研]
永久联系微信:4550060

算法思想是：从主串 S 的第一个字符起，与模式串 T 的第一个字符比较，若相等，则继续逐个比较后续字符；否则从主串的下一个字符起，再重新和模式串 T 的字符比较；以此类推，直至模式串 T 中的每个字符依次和主串 S 中的一个连续的字符序列相等，则称匹配成功，函数值为与模式串 T 中第一个字符相等的字符在主串 S 中的序号，否则称匹配不成功，函数值为零。

图 4.2 展示了模式串 T='abcac'和主串 S 的匹配过程。

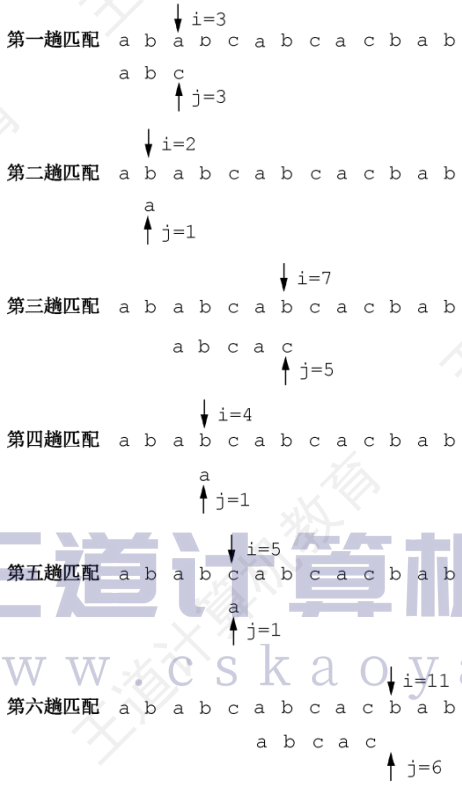


图 4.2　简单模式匹配算法举例

在简单模式匹配算法中，设主串和模式串的长度分别为 n 和 m（$n \gg m$），则最多需要进行 $n-m+1$ 趟匹配，每趟最多需要进行 m 次比较，最坏时间复杂度为 $O(nm)$。例如，当模式串为'0000001'而主串为'00000000000000000000000000000000000000000000 000001'时，由于模式串中的前 6 个字符均为'0'，主串中的前 45 个字符均为'0'，每趟匹配都是比较到模式串中的最后一个字符时才发现不等，整个匹配过程中指针 i 需要回溯 39 次，总比较次数为 40×7＝280 次。

4.2.2　串的模式匹配算法——KMP 算法

在图 4.2 的第三趟匹配过程中，i=7、j=5 的字符比较，结果不等，于是又从 i=4、j=1 重新开始比较。然而，仔细观察会发现，i=4 和 j=1、i=5 和 j=1 以及 i=6 和 j=1 这三次比较都是不必进行的。从第三趟部分匹配的结果可知，主串的第 4 个、第 5 介、第 6 个字符是'b'、'c'、'a'（模式串的第 2、第 3、第 4 个字符），因为模式串的第 1 个字符是'a'，所以再和这三个字符进行比较纯属多余，而只需将模式串向右滑动三个字符的位置，再进行 i=7、j=2 的比较即可。

在简单模式匹配算法中，每趟匹配失败都是模式串向右滑动一位后从头开始比较的。而某趟已匹配相等的字符序列是模式串的某个前缀，因此可从分析模式串本身的结构着手，若已匹配相等的前缀序列中有某个后缀正好是模式串的前缀，则可将模式串向右滑动到与这些相等字符对齐

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

的位置（也是后面手算 next 数组的依据），主串指针 i 无须回溯，并从该位置开始继续比较。而模式串向右滑动位数的计算仅与模式串本身的结构有关，与主串无关。

1．KMP 算法的原理

要了解模式串的结构，首先要弄清楚几个概念：前缀、后缀和部分匹配值。前缀是指除最后一个字符外，字符串的所有头部子串；后缀是指除第一个字符外，字符串的所有尾部子串；部分匹配值则是指字符串的前缀和后缀的最长相等前后缀长度。下面以'ababa'为例进行说明：

- 'a'的前缀和后缀都为空集，最长相等前后缀长度为 0。
- 'ab'的前缀为{a}，后缀为{b}，{a}∩{b}=∅，最长相等前后缀长度为 0。
- 'aba'的前缀为{a,ab}，后缀为{a,ba}，{a,ab}∩{a,ba}={a}，最长相等前后缀长度为 1。
- 'abab'的前缀{a,ab,aba}∩后缀{b,ab,bab}={ab}，最长相等前后缀长度为 2。
- 'ababa'的前缀{a,ab,aba,abab}∩后缀{a,ba,aba,baba}={a,aba}，公共元素有两个，最长相等前后缀长度为 3。

因此，模式串'ababa'的部分匹配值为 00123。

这个部分匹配值有什么作用呢？

回到最初的问题，主串为'ababcabcacbab'，模式串为'abcac'。

利用上述方法容易求出模式串'abcac'的部分匹配值为 00010，将部分匹配值写成数组形式，就得到了部分匹配值（Partial Match，PM）的表。

编号	1	2	3	4	5
S	a	b	c	a	c
PM	0	0	0	1	0

王道计算机教育 www.cskaoyan.com

下面用 PM 表来进行字符串匹配：

主串　a b a b c a b c a c b a b
模式串　a b c

第一趟匹配过程：

发现 c 与 a 不匹配，前面的 2 个字符'ab'是匹配的，查表可知，最后一个匹配字符 b 对应的部分匹配值为 0，因此按照下面的公式算出模式串需要的右滑位数：

右滑位数 = 已匹配的字符数 − 对应的部分匹配值

因为 2 − 0 = 2，所以将模式串向右滑动 2 位如下，进行第二趟匹配：

主串　a b a b c a b c a c b a b
模式串　　a b c a c

第二趟匹配过程：

发现 c 与 b 不匹配，前面的 4 个字符'abca'是匹配的，查表可知，最后一个匹配字符 a 对应的部分匹配值为 1，4 − 1 = 3，将模式串向右滑动 3 位如下，进行第三趟匹配：

主串　a b a b c a b c a c b a b
模式串　　a b c a c

第三趟匹配过程：

模式串全部比较完成，匹配成功。整个匹配过程中，主串始终没有回退，所以 KMP 算法可在 $O(n+m)$的时间数量级上完成串的模式匹配操作，大大提高了匹配效率。

配套课程公众号[考机研]
永久联系微信:4550060

某趟发生失配时，若已匹配相等的序列中没有相等的前后缀，则对应的部分匹配值为0，此时滑动的位数最大，直接将模式串首字符向右滑动到主串当前位置进行下一趟比较；若已匹配相等的序列中存在最大相等前后缀（可理解为首尾重合），则将模式串向右滑动到和主串中该相等后缀对齐（这些重合的字符下一趟显然无需再比较），然后从主串当前位置进行下一趟比较。两种情况的模式串右滑位数都等于“已匹配的字符数 - 对应的部分匹配值”。

还有一种特例，在上述举例中并未出现，当某趟第一个字符比较就失配时，应如何处理呢？此时，应让模式串向右滑动一位，再从主串当前位置的下一位开始比较。

2．next数组的手算方法

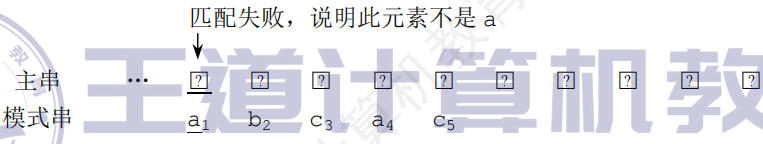
在实际的匹配过程中，模式串在内存中是不会滑动的，发生变化的是指针，前面的举例只是手动模拟KMP算法的过程，也是为了让读者更为形象地进行理解。

命题追踪 ▶ KMP算法中指针变化、比较次数的分析（2015、2019）

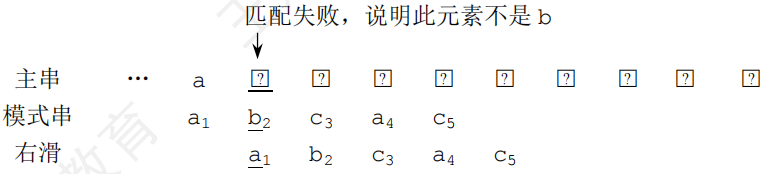
每趟匹配失败时，只有模式串指针i在变化，主串指针j不会回溯，为此可以定义一个next数组，next[j]的含义是当模式串的第j个字符失配时，跳到next[j]位置继续比较。

下面给出一种求next数组的手算方法，仍以模式串'abcac'为例。

第1个字符失配时，令next[1]=0，然后指针i和j同时加1，即下次将模式串的第1个位置与主串当前位置的下一位置进行比较（注意，图中的下标为模式串编号）。

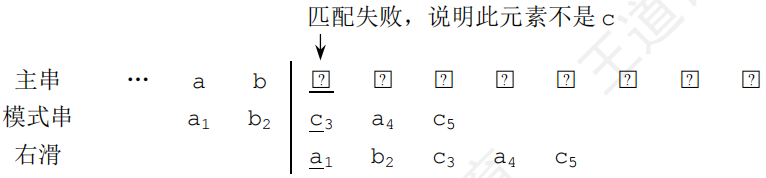


第2个字符失配时，令next[2]=1，模式串的下次比较位置为1，相当于向右滑动1位。注，模式串的next[1]=0、next[2]=1都是固定不变的。

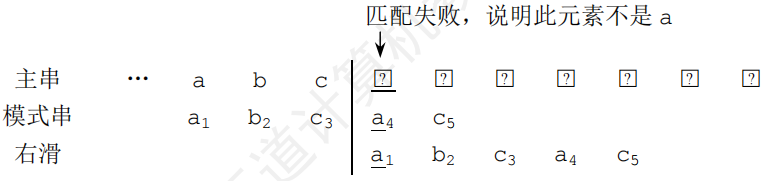


在后面的手算过程中，在不匹配的位置前画一条分界线，模式串一步一步往后退，直到分界线之前能对上（首尾重合），或模式串完全跨过分界线为止。

第3个字符失配时，模式串的下次比较位置为1，即next[3]=1，相当于向右滑动2位。

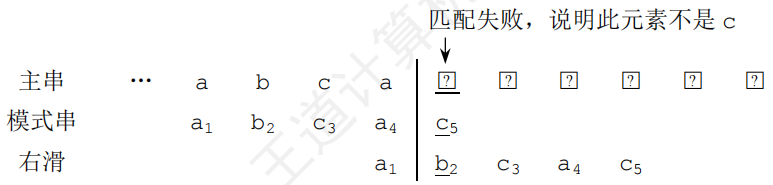


第4个字符失配时，模式串的下次比较位置为1，即next[4]=1，相当于向右滑动3位。



王道计算机教育 www.wdkaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

第 5 个字符失配时，模式串的下次比较位置为 2，即 next[5]=2，相当于向右滑动 3 位。



next 数组和 PM 表的关系是怎样的?

通过上述举例，可以推理出 next 数组和 PM 表之间的关系：

next[j] = j-右滑位数 = j-(已匹配的字符数[①]-对应的部分匹配值)
= j-[(j-1)-PM[j-1]]
= PM[j-1]+1

根据上述结论，将模式串'abcac'的 PM 表右移一位，并整体加 1，就得到了模式串'abcac'对应的 next 数组，通过与前面手算的结果比较，可以验证上述结论。

编号	1	2	3	4	5
S	a	b	c	a	c
next	0	1	1	1	2

我们注意到：

1）第一个元素右滑以后空缺的用 0 来填充，因为若是第一个元素匹配失败，则需要将主串指针和模式串指针同步右移一位，从而不需要计算模式串指针移动的位数。

2）最后一个元素在右滑的过程中溢出，因为原来的模式串中，最后一个元素的部分匹配值是其下一个元素使用的，但显然已没有下一个元素，所以可以舍去。

注　意

上述 KMP 算法的举例中，都假设串的编号是从 1 开始的；若串的编号是从 0 开始的，则 next 数组需要整体减 1。

*3. next 数组的推理公式

如何推理 next 数组的一般公式？设主串为'$s_1s_2\cdots s_n$'，模式串为'$p_1p_2\cdots p_m$'，当主串的第 i 个字符与模式串的第 j 个字符失配时，应让主串当前位置与模式串的哪个字符进行比较?

假设此时应与模式串的第 k（k<j）个字符进行比较，则模式串的前 k-1 个字符的子串必须满足下列条件，且不可能存在 k′>k 满足下列条件：

'$p_1p_2\cdots p_{k-1}$'='$p_{j-k+1}p_{j-k+2}\cdots p_{j-1}$'

若存在满足如上条件的子串，则发生失配时，仅需将模式串的第 k 个字符和主串的第 i 个字符对齐，此时模式串的前 k-1 个字符的子串必定与主串的第 i 个字符之前长度为 k-1 的子串相等，因此，只需从模式串的第 k 个字符与主串的第 i 个字符进行比较即可，如图 4.3 所示。

主串	s_1	…	…	…	…	…	s_{i-k+1}	…	s_{i-1}	s_i	…	…	…	…	s_n
子串			p_1	…	p_{k-1}	…	p_{j-k+1}	…	p_{j-1}	p_j	…	p_m			
右滑							p_1	…	p_{k-1}	p_k	…	…	p_m		

图 4.3　模式串右滑到合适位置（阴影对齐部分表示上下字符相等）

① 注意，当模式串匹配到位置 j 失配时，已匹配成功的字符数为 j-1。

配套课程公众号[考机研]
永久联系微信:4550060

当模式串已匹配相等序列中不存在满足上述条件的子串时（可视为 k=1），显然应让主串的第 i 个字符和模式串的第 1 个字符进行比较。

当模式串的第 1 个字符（j=1）与主串的第 i 个字符发生失配时，规定 next[1]=0。

通过上述分析可以得出 next 函数的公式：

$$next[j]=\begin{cases}0, & j=1\\ \max\{k \mid 1<k<j \text{且} 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\}, & \text{当此集合不为空时}\\ 1, & \text{其他情况}\end{cases}$$

要用代码来实现，难度貌似还不小，下面来尝试推理求解的科学步骤。

首先由公式可知

$$next[1]=0$$

设 next[j]=k，此时 k 应满足的条件在上文中已描述。

此时 next[j+1]=? 可能有两种情况：

（1）若 $p_k=p_j$，则表明在模式串中

$$'p_1\cdots p_{k-1}p_k'='p_{j-k+1}\cdots p_{j-1}p_j'$$

且不可能存在 k′>k 满足上述条件，此时 next[j+1]=k+1，即

$$next[j+1]=next[j]+1$$

（2）若 $p_k\neq p_j$，则表明在模式串中

$$'p_1\cdots p_{k-1}p_k'\neq'p_{j-k+1}\cdots p_{j-1}p_j'$$

此时可将求 next 函数值的问题视为一个模式匹配问题。用前缀 $p_1\cdots p_k$ 去与后缀 $p_{j-k+1}\cdots p_j$ 匹配，当 $p_k\neq p_j$ 时，应将 $p_1\cdots p_k$ 向右滑动至用第 next[k]个字符与 p_j 进行比较，若 $p_{next[k]}$ 与 p_j 仍不匹配，则需要寻找长度更短的相等前后缀，下一步继续用 $P_{next[next[k]]}$ 与 p_j 进行较，以此类推，直到找到某个更小的 k′=next[next…[k]]（1<k′<k<j），满足条件

$$'p_1\cdots p_{k'}'='p_{j-k'+1}\cdots p_j'$$

则 next[j+1]=k′+1。

也可能不存在任何 k′满足上述条件，即不存在长度更短的相等前后缀，令 next[j+1]=1。

理解起来有点儿费劲？下面举一个简单的例子。

图 4.4 的模式串中已求得 6 个字符的 next 值，现求 next[7]，因为 next[6]=3，又 $p_6\neq p_3$，所以需要比较 p_6 和 p_1（因 next[3]=1），$p_6\neq p_1$，而 next[1]=0，因此 next[7]=1；求 next[8]，因为 $p_7=p_1$，所以 next[8]=next[7]+1=2；求 next[9]，因为 $p_8=p_2$，所以 next[9]=3。

j	1	2	3	4	5	6	7	8	9
模式串	a	b	a	a	b	c	a	b	a
next[j]	0	1	1	2	2	3	?	?	?

图 4.4　求模式串的 next 值

***4．KMP 算法的实现**

通过上述分析写出求 next 值的程序如下：

```
void get_next(SString T,int next[]){
    int i=1,j=0;
    next[1]=0;
    while(i<T.length){
        if(j==0||T.ch[i]==T.ch[j]){
            ++i; ++j;
            next[i]=j;  //若 pi=pj，则 next[j+1]=next[j]+1
        }
```

配套课程公众号[考机研]
永久联系微信:4550060

```
            else
                j=next[j];  //否则令 j=next[j]，循环继续
        }
    }
```

计算机执行起来效率很高，但需要手工计算时，仍然采用前面的方法。

与 next 数组的求解相比，KMP 的匹配算法相对要简单很多，它在形式上与简单的模式匹配算法很相似。不同之处仅在于当匹配过程产生失配时，指针 i 不变，指针 j 退回到 next[j]的位置并重新进行比较，且当指针 j 为 0 时，指针 i 和 j 同时加 1。也就是说，若主串的第 i 个位置和模式串的第 1 个字符不等，则应从主串的第 i+1 个位置开始匹配。具体代码如下：

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&&j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            ++i; ++j;              //继续比较后继字符
        }
        else
            j=next[j];             //模式串向右滑动
    }
    if(j>T.length)
        return i-T.length;         //匹配成功
    else
        return 0;
}
```

尽管普通模式匹配的时间复杂度是 $O(mn)$，KMP 算法的时间复杂度是 $O(m+n)$，但在一般情况下，普通模式匹配的实际执行时间复杂度近似为 $O(m+n)$，因此至今仍被采用。KMP 算法仅在主串与子串有很多“部分匹配”时才显得比普通算法快，其主要优点是主串不回溯。

4.2.3　KMP 算法的进一步优化

命题追踪 ▶▶ nextval 数组的计算（2024）

前面定义的 next 数组在某些情况下尚有缺陷，还可以进一步优化。如图 4.5 所示，模式串'aaaab'在和主串'aaabaaaab'进行匹配时。

主串	a	a	a	b	a	a	a	a	b
模式串	a	a	a	a	b				
j	1	2	3	4	5				
next[j]	0	1	2	3	4				
nextval[j]	0	0	0	0	4				

图 4.5　KMP 算法进一步优化示例

当 i=4、j=4 时，s_4 跟 p_4（b≠a）失配，若用之前的 next 数组，则还需要进行 s_4 与 p_3、s_4 与 p_2、s_4 与 p_1 这 3 次比较。事实上，因为 $p_{next[4]=3}=p_4=a$、$p_{next[3]=2}=p_3=a$、$p_{next[2]=1}=p_2=a$，显然后面 3 次用一个和 p_4 相同的字符跟 s_4 比较毫无意义，必然失配。那么问题出在哪里呢？

问题在于不应该出现 $p_j=p_{next[j]}$。理由是：当 $p_j \neq s_i$ 时，下次匹配必然是 $p_{next[j]}$ 跟 s_i 比较，若 $p_j=p_{next[j]}$，则相当于拿一个和 p_j 相等的字符跟 s_i 比较，这必然导致继续失配，这样的比较毫无意义。若出现 $p_j=p_{next[j]}$，则如何处理呢？

若出现 $p_j=p_{next[j]}$，则需要再次递归，将 next[j]修正为 next[next[j]]，直至两者不相等为止，更新后的数组命名为 nextval。计算 next 数组修正值的算法如下，此时匹配算法不变。

```
void get_nextval(SString T,int nextval[]){
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        int i=1, j=0;
        nextval[1]=0;
        while(i<T.length){
            if(j==0||T.ch[i]==T.ch[j]){
                ++i; ++j;
                if(T.ch[i]!=T.ch[j])  nextval[i]=j;
                else  nextval[i]=nextval[j];
            }
            else
                j=nextval[j];
        }
    }
```

KMP算法对于初学者来说可能不太容易掌握，建议读者结合王道课程来理解。

4.2.4 本节试题精选

一、单项选择题

01. 设有两个串S_1和S_2，求S_2在S_1中首次出现的位置的运算称为（ ）。

A. 求子串　　B. 判断是否相等　　C. 模式匹配　　D. 连接

02. KMP算法的特点是在模式匹配时，指示主串的指针（ ）。

A. 不会变大　　B. 不会变小　　C. 都有可能　　D. 无法判断

03. 设主串的长度为n，子串的长度为m，则简单的模式匹配算法的时间复杂度为（ ），KMP算法的时间复杂度为（ ）。

A. $O(m)$　　B. $O(n)$　　C. $O(mn)$　　D. $O(m+n)$

04. 在KMP算法中，用next数组存放模式串的部分匹配信息，当模式串位j与主串位i比较时，两个字符不相等，则j的位移方式是（ ）。

A. j=0　　B. j=j+1　　C. j不变　　D. j=next[j]

05. 在KMP算法中，用next数组存放模式串的部分匹配信息，当模式串位j与主串位i比较时，两个字符不相等，则i的位移方式是（ ）。

A. i=next[i]　　B. i不变　　C. i=0　　D. i=i+1

06. 串'ababaaababaa'的next数组为（ ）。

A. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9　　B. 0, 1, 2, 1, 2, 1, 1, 1, 1, 2, 1, 2

C. 0, 1, 1, 2, 3, 4, 2, 2, 3, 4, 5, 6　　D. 0, 1, 2, 3, 0, 1, 2, 3, 2, 2, 3, 4, 5

07. 串'ababaaababaa'的next数组为（ ）。

A. −1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 8, 8　　B. −1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1

C. −1, 0, 0, 1, 2, 3, 1, 1, 2, 3, 4, 5　　D. −1, 0, 1, 2, −1, 0, 1, 2, 1, 1, 2, 3

08. 设主串S='aabaaaba'，模式串T='aaab'，采用KMP算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

A. 10　　B. 9　　C. 8　　D. 7

09. 设主串S='aabaaaba'，模式串T='aaab'，采用改进后的KMP算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

A. 9　　B. 8　　C. 7　　D. 6

10. KMP算法使用nextval数组进行模式匹配，模式串为S='ababa**a**a'，当主串中的某字符与S中的第6个字符失配时，S向右滑动的距离是（ ）。

A. 1　　B. 2　　C. 3　　D. 4

11. 【2015统考真题】已知字符串s为'abaabaabacacaabaabcc'，模式串t为'abaabc'。

配套课程公众号[考机研]
永久联系微信:4550060

采用 KMP 算法进行匹配，第一次出现“失配”（s[i]≠t[j]）时，i=j=5，则下次开始匹配时，i 和 j 的值分别是（　）。

A. i=1，j=0　　B. i=5，j=0　　C. i=5，j=2　　D. i=6，j=2

12. 【2019 统考真题】设主串 T='abaabaabcabaabc'，模式串 S='abaabc'，采用 KMP 算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（　）。

A. 9　　B. 10　　C. 12　　D. 15

13. 【2024 统考真题】KMP 算法使用修正后的 next 数组进行模式匹配，模式串为 S='aabaab'，当主串的某个字符与 S 的某个字符失配时，S 向右滑动的最长距离是（　）。

A. 5　　B. 4　　C. 3　　D. 2

二、综合应用题

01. 在字符串模式匹配的 KMP 算法中，求模式的 next 数组值的定义如下：

$$\text{next}[j]=\begin{cases}0, & j=1\\ \max\{k \mid 1<k<j \text{且} 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\}, & \text{集合不为空}\\ 1, & \text{其他情况}\end{cases}$$

1）当 j=1 时，为什么要取 next[1]=0？

2）为什么要取 max{k}，k 最大是多少？

3）其他情况是什么情况，为什么取 next[j]=1？

02. 设有字符串 S='aabaabaabaac'，P='aabaac'。

1）求出 P 的 next 数组。

2）若 S 作主串，P 作模式串，试给出 KMP 算法的匹配过程。

4.2.5　答案与解析

一、单项选择题

01. C

求子串操作是从串 S 中截取第 i 个字符起长度为 l 的子串，选项 A 错误。选项 B、D 明显错误。

02. B

在 KMP 算法的比较过程中，主串不会回溯，所以主串的指针不会变小。

03. C、D

尽管实际应用中，一般情况下简单的模式匹配算法的时间复杂度近似为 $O(m+n)$，但它的理论时间复杂度还是 $O(mn)$。KMP 算法的时间复杂度为 $O(m+n)$。

04. D

在 KMP 算法中，当主串的第 i 个字符和模式串的第 j 个字符不匹配时，主串的位指针 i 不变，将主串的第 i 个字符与模式串的第 next[j]个字符比较，即 j=next[j]。

05. B

在 KMP 算法中，当主串的第 i 个字符和模式串的第 j 个字符失配时，主串指针 i 不回溯。

06. C

本题采用先求串 S='ababaaababaa'的部分匹配值，再求 next 数组的方法。

- 'a'的前后缀都为空，最长相等前后缀长度为 0。
- 'ab'的前缀{a}∩后缀{b}=∅，最长相等前后缀长度为 0。
- 'aba'的前缀{a,ab}∩后缀{a,ba}={a}，最长相等前后缀长度为 1。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

- 'abab'的前缀{a,ab,aba}∩后缀{b,ab,bab}={ab}，最长相等前后缀长度为2。
- ……

依次求出的部分匹配值见下表第3行，将其整体右移一位，低位用-1填充，见下表第4行。

编号	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
PM	0	0	1	2	3	1	1	2	3	4	5	6
next	-1	0	0	1	2	3	1	1	2	3	4	5

选项中next[1]等于0，所以将next数组整体加1。

07. C

解析见上题。读者也可尝试用前面介绍的手算方法。

注意，next数组是否整体加1都正确，需根据题意具体分析。

08. B

假设位序从1开始的，手工计算出T的next数组如下。

编号	1	2	3	4
S	a	a	a	b
next	0	1	2	3

采用KMP匹配算法时：第一趟，经过3次比较后，发现S[3]≠T[3]；第二趟，用S[3]和T[2]（next[3]=2）比较，不相等；第三趟，用S[3]和T[1]（next[2]=1）比较，不相等；第四趟，next[1]=0，因此开始用S[4]和T[1]比较，经过4次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟		a	a	a	b			
第三趟			a	a	a	b		
第四趟				a	a	a	b	

总比较次数为3+1+1+4=9。

09. C

假设位序从1开始的，计算出T的nextval数组如下。

编号	1	2	3	4
S	a	a	a	b
nextval	0	0	0	3

采用改进的KMP匹配算法时：第一趟，经过3次比较后，发现S[3]≠T[3]；第二趟，nextval[3]=0，因此开始用S[4]和T[1]比较，经过4次比较后，匹配成功。整个匹配过程如下。

	a	a	b	a	a	a	b	a
第一趟	a	a	a	b				
第二趟				a	a	a	b	

总比较次数为3+4=7。

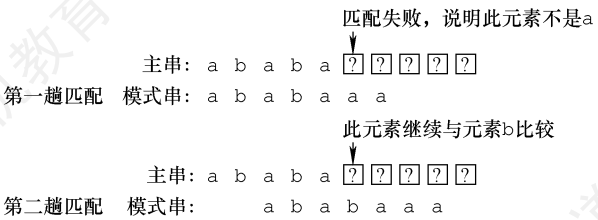
10. B

假设位序从0开始的，计算出nextval数组。当比较到S[j]失配时，模式串向右滑动的距离为j-nextval[j]（0≤j≤6），由下图可知，当j=5时向右滑动的距离为5-3=2。

配套课程公众号[考机研]
永久联系微信:4550060

编号 j	0	1	2	3	4	5	6
S[j]	a	b	a	b	a	a	a
next[j]	-1	0	0	1	2	3	1
nextval[j]	-1	0	-1	0	-1	3	1
j-nextval[j]	1	1	3	3	5	2	5

此外，若对 KMP 算法很熟悉，则此类题也可直接动手模拟，而不需要求 nextval 数组，与第 6 个字符匹配失败，说明前面的 ababa 匹配成功，S 可向右滑动 2 位继续尝试匹配。



11. C

由题中“失配 s[i]≠t[j]时，i=j=5”，可知题中的主串和模式串的位序都是从 0 开始的（要注意灵活应变）。按照 next 数组生成算法，对于 t 有

编号	0	1	2	3	4	5
t	a	b	a	a	b	c
next	-1	0	0	1	1	2

发生失配时，主串指针 i 不变，子串指针 j 回退到 next[j]位置重新比较，当 s[i]≠t[j]时，i=j=5，由 next 表得知 next[j]=next[5]=2（位序从 0 开始）。因此，i=5，j=2。

12. B

假设位序从 0 开始的，按照 next 数组生成算法，对于 S 有

编号	0	1	2	3	4	5
S	a	b	a	a	b	c
next	-1	0	0	1	1	2

第一趟连续比较 6 次，在模式串的 5 号位和主串的 5 号位匹配失败，模式串的下一个比较位置为 next[5]，即下一次比较从模式串的 2 号位和主串的 5 号位开始，然后直到模式串的 5 号位和主串的 8 号位匹配，第二趟比较 4 次，匹配成功。单个字符的比较次数为 10 次。

13. A

假设位序从 0 开始的，计算出 nextval 数组。当比较到 S[j]失配时，模式串向右滑动的距离为 j－nextval[j]（0≤j≤5），由下图可知，当 j＝4 时向右滑动的距离最长，此时距离为 5。

编号 j	0	1	2	3	4	5
S[j]	a	a	b	a	a	b
next[j]	-1	0	1	0	1	2
nextval[j]	-1	-1	1	-1	-1	1
j-nextval[j]	1	2	1	4	5	4

配套课程公众号[考机研]
永久联系微信:4550060

二、综合应用题

01. 【解答】

1）当模式串的第1个字符与主串的当前字符比较不相等时，next[1]=0，表示模式串应右滑一位，主串当前指针后移一位，再和模式串的第1个字符进行比较。

2）当主串的第 i 个字符与模式串的第 j 个字符失配时，主串 i 不回溯，则假定模式串的第 k 个字符与主串的第 i 个字符比较，k 值应满足条件 $1<k<j$ 且 $'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'$，即 k 为模式串的下次比较位置。k 值可能有多个，为了不使向右滑动丢失可能的匹配，右滑距离应该取最小，因为 j-k 表示右滑的距离，所以取 max{k}。k 的最大值为 j-1。

3）除上面两种情况外，发生失配时，主串指针 i 不回溯，在最坏情况下，模式串从第1个字符开始与主串的第 i 个字符比较。

02. 【解答】

1）P='aabaac'，按照 next 数组生成算法，对于 P 有：

① 设 next[1]=0，next[2]=1。

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1				

② j=3 时 k=next[j-1]=next[2]=1，观察 S[j-1](S[2])与 S[k](S[1])是否相等，S[2]=a，S[1]=a，S[2]=S[1]，所以 next[j]=k+1=2。

```
   ↓j-1=2
a  a  b  a  a  c
a  a  b  a  a  c
↑k=1
```

③ j=4 时 k=next[j-1]=next[3]=2，观察 S[j-1](S[3])与 S[k](S[2])是否相等，S[3]=b，S[2]=a，S[3]!=S[2]。

```
      ↓j-1=3
a  a  b  a  a  c
   a  a  b  a  a  c
   ↑k=2
```

此时 k=next[k]=1，观察 S[3]与 S[k](S[1])是否相等，S[3]=b，S[1]=a，S[3]!=S[1]。k=next[k]=0，因为 k=0，所以 next[j]=1。

```
      ↓j-1=3
a  a  b  a  a  c
      a  a  b  a  a  c
↑k=0
```

④ j=5 时 k=next[j-1]=next[4]=1，观察 S[j-1](S[4])与 S[k](S[1])是否相等，S[4]=a，S[1]=a，S[4]=S[1]，所以 next[j]=k+1=2。

```
         ↓j-1=4
a  a  b  a  a  c
         a  a  b  a  a  c
         ↑k=1
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

⑤ j=6 时 k=next[j-1]=next[5]=2，观察 S[j-1](S[5])与 S[k](S[2])是否相等，S[5]=a，S[2]=a，S[5]=S[2]，所以 next[j]=k+1=3。

```
                  ↓j-1=5
  a   a   b   a   a   c
          a   a   b   a   a   c
              ↑k=2
```

最后的结果为

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1	2	1	2	3

也可以通过求部分匹配值表的方法来求 next 数组。

2）利用 KMP 算法的匹配过程如下。

第一趟：从主串和模式串的第 1 个字符开始比较，失配时 i=6，j=6。

主串	a	a	b	a	a	b	a	a	b	a	a	c
	a	a	b	a	a	c						

第二趟：next[6]=3，主串当前位置和模式串的第 3 个字符继续比较，失配时 i=9，j=6。

主串	a	a	b	a	a	b	a	a	b	a	a	c
			a	a	b	a	a	c				

第三趟：next[6]=3，主串当前位置和模式串的第 3 个字符继续比较，匹配成功。

主串	a	a	b	a	a	b	a	a	b	a	a	c
							a	a	b	a	a	c

归纳总结

学习 KMP 算法时，应从分析暴力法的弊端入手，思考如何去优化它。实际上，已匹配相等的序列就是模式串的某个前缀，因此每次回溯就相当于模式串与模式串的某个前缀在比较，这种频繁的重复比较是其效率低的原因。这时，可从分析模式串本身的结构入手，以便得知当匹配到某个字符不等时，应该向右滑动到什么位置，即已匹配相等的某个前缀若与模式串首尾重合，则对齐它们，对齐部分显然无需再比较，然后直接从主串的当前位置继续开始比较。

思维拓展

编程实现：模式串在主串中有多少个完全匹配的子串？注意，统考不考 KMP 算法题。

配套课程公众号[考机研]
永久联系微信:4550060